



MÓDULO PROYECTO

CFGS Desarrollo de Aplicaciones WEB
Informática y Comunicaciones

HippoTrack

Tutor individual: José Antonio

Tutor colectivo: José Luis Román

Año: 2026

Fecha de presentación: 26/05/2026

Nombre y Apellidos: Javier Centeno López
Email: javier.cenlop@educa.jcyl.es

**Foto actual
del alumno**

Tabla de contenido

1	Identificación proyecto	3
2	Organización de la memoria	4
3	Descripción general del proyecto	5
3.1	Objetivos	5
3.2	Cuestiones metodológicas	5
3.3	Entorno de trabajo (tecnologías de desarrollo y herramientas)	6
4	Descripción general del producto	8
4.1	Visión general del sistema: límites del sistema, funcionalidades básicas, usuarios y/o otros sistemas con los que pueda interactuar.	8
4.2	Descripción breve de métodos, técnicas o arquitecturas(m/t/a) utilizadas.	8
4.3	Despliegue de la aplicación indicando plataforma tecnológica, instalación de la aplicación y puesta en marcha.....	9
5	Planificación y presupuesto	11
6	Documentación Técnica: análisis, diseño, implementación y pruebas.	13
6.1	Especificación de requisitos	13
6.2	Análisis del sistema	13
6.3	Diseño del sistema:	14
6.3.1	Diseño de la Base de Datos	14
6.3.2	Diseño de la Interfaz de usuario.	15
6.3.3	Diseño de la Aplicación.	16
6.4	Implementación:	18
6.4.1	Entorno de desarrollo.	18
6.4.2	Estructura del código.	18
6.4.3	Cuestiones de diseño e implementación reseñables.	18
6.5	Pruebas.	19
7	Manuales de usuario	20
7.1	Manual de usuario	20

7.2	Manual de instalación.....	28
8	Conclusiones y posibles ampliaciones	32
9	Bibliografía	33
10	Anexos.....	¡Error! Marcador no definido.

1 Identificación proyecto

El presente Trabajo de Fin de Grado consiste en el desarrollo de una aplicación web orientada a la gestión y visualización de carreras de caballos, incorporando además un sistema de apuestas para los usuarios registrados.

La aplicación permite consultar información relacionada con hipódromos, carreras, caballos, entrenadores, jinetes y resultados, ofreciendo una plataforma centralizada para el seguimiento de competiciones hípcas. Además, los usuarios pueden registrarse, iniciar sesión y realizar apuestas sobre los participantes de cada carrera.

El sistema ha sido desarrollado utilizando Django como framework principal para el backend y PostgreSQL como sistema gestor de base de datos. La arquitectura del proyecto sigue el patrón MVT (Model-View-Template), favoreciendo una correcta separación entre lógica, datos e interfaz.

La base de datos diseñada para el proyecto incluye entidades relacionadas con usuarios, carreras, participantes, apuestas, caballos, jinetes, entrenadores y resultados, permitiendo gestionar de forma estructurada toda la información del sistema.

Asimismo, el proyecto incorpora scripts de procesamiento y organización de datos procedentes de diferentes fuentes relacionadas con carreras hípcas, automatizando la carga y estructuración de información dentro de la base de datos.

El objetivo principal del proyecto es aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo y durante el periodo de formación en empresa, integrando tecnologías web modernas, bases de datos relacionales, autenticación de usuarios y desarrollo backend completo dentro de una aplicación funcional y escalable.

2 Organización de la memoria

La presente memoria se encuentra estructurada en diferentes apartados con el objetivo de describir de forma clara y ordenada todo el proceso de desarrollo del proyecto, desde la planificación inicial hasta la implementación final de la aplicación.

En primer lugar, se realiza una descripción general del proyecto, indicando los objetivos principales, la metodología seguida y las tecnologías empleadas durante el desarrollo. Posteriormente, se presenta una visión general del producto desarrollado, detallando las funcionalidades principales del sistema, los usuarios que interactúan con la aplicación y la arquitectura utilizada.

A continuación, se incluye la planificación del proyecto y una estimación presupuestaria relacionada con el desarrollo de la aplicación.

La parte principal de la memoria corresponde a la documentación técnica, donde se describen aspectos relacionados con el análisis, diseño, implementación y pruebas del sistema. Dentro de esta sección se detallan tanto la estructura de la base de datos como el diseño de la interfaz de usuario y la organización interna de la aplicación.

Posteriormente, se incorporan los manuales de usuario e instalación, cuyo objetivo es facilitar tanto el uso de la plataforma como su puesta en funcionamiento en un entorno de desarrollo o producción.

Finalmente, la memoria concluye con una valoración general del trabajo realizado, posibles ampliaciones futuras del sistema, las referencias bibliográficas utilizadas y los anexos complementarios que apoyan la documentación del proyecto.

La estructura seguida en esta memoria se basa en el esquema propuesto para el módulo de Proyecto del CFGS de Desarrollo de Aplicaciones Web.

3 Descripción general del proyecto

3.1 Objetivos

El principal objetivo de este proyecto es desarrollar una aplicación web funcional orientada a la gestión y visualización de carreras de caballos, incorporando un sistema de apuestas para usuarios registrados.

La aplicación pretende centralizar toda la información relacionada con competiciones hípicas, permitiendo consultar carreras, participantes, caballos, jinetes, entrenadores y resultados desde una única plataforma accesible mediante navegador web.

Además, el proyecto tiene como finalidad aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Web y durante el periodo de formación en empresa, integrando tecnologías actuales utilizadas en el desarrollo profesional de aplicaciones web.

Entre los objetivos específicos del proyecto destacan los siguientes:

- Diseñar e implementar una base de datos relacional capaz de gestionar correctamente toda la información del sistema.
- Desarrollar un sistema de autenticación y gestión de usuarios.
- Permitir la visualización dinámica de carreras y participantes.
- Implementar un sistema de apuestas asociado a cada carrera.
- Gestionar relaciones complejas entre entidades como caballos, jinetes, entrenadores y resultados.
- Aplicar buenas prácticas de desarrollo utilizando arquitectura MVT.
- Automatizar la organización y procesamiento de datos mediante scripts en Python.
- Diseñar una interfaz clara, intuitiva y accesible para el usuario.
- Mejorar las capacidades de desarrollo backend y gestión de bases de datos adquiridas durante la formación académica.

3.2 Cuestiones metodológicas

Para el desarrollo del proyecto se ha seguido una metodología incremental basada en el desarrollo progresivo de funcionalidades. Este enfoque ha permitido construir la aplicación por fases, validando cada parte del sistema antes de continuar con nuevas implementaciones.

El desarrollo comenzó con el análisis de requisitos y el diseño inicial de la base de datos, definiendo las entidades principales del sistema y sus relaciones. Posteriormente, se implementó la estructura del proyecto utilizando Django como framework principal.

A medida que avanzaba el proyecto, se fueron incorporando funcionalidades adicionales como:

- registro e inicio de sesión de usuarios,
- visualización de carreras,
- sistema de apuestas,
- perfiles de usuario,
- consultas estadísticas,
- y procesamiento automatizado de datos.

Durante el desarrollo también se realizaron pruebas continuas para detectar errores de funcionamiento, problemas de integridad en la base de datos y fallos relacionados con autenticación o navegación dentro de la aplicación.

La metodología utilizada ha permitido mantener una evolución organizada del proyecto, facilitando la corrección de errores y la ampliación progresiva del sistema.

3.3 Entorno de trabajo (tecnologías de desarrollo y herramientas)

El proyecto ha sido desarrollado utilizando diferentes tecnologías y herramientas orientadas al desarrollo web backend, gestión de bases de datos y procesamiento de información.

Las principales tecnologías utilizadas han sido:

- Python como lenguaje principal de programación.
- Django como framework backend para el desarrollo de la aplicación web.
- PostgreSQL como sistema gestor de base de datos relacional.
- HTML y CSS para la estructura y diseño visual de las interfaces.
- JavaScript para determinadas funcionalidades dinámicas en el cliente.
- SQL para la creación y gestión de la base de datos.

Además, se utilizaron distintas herramientas de desarrollo:

- Visual Studio Code como entorno de desarrollo principal.
- Git como sistema de control de versiones.
- Navegadores web modernos para pruebas y depuración de la interfaz.
- Scripts en Python para el tratamiento y reorganización de datos relacionados con las carreras hípcas.
- Entorno virtual de Python para la gestión de dependencias del proyecto.

A diferencia de tecnologías como PHP, que suelen ejecutarse junto a servidores como Apache, o aplicaciones Java desplegadas en servidores como Apache Tomcat, Django incorpora un servidor de desarrollo integrado propio.

Durante el desarrollo del proyecto se utilizó dicho servidor, encargado de gestionar las peticiones HTTP realizadas desde el navegador y ejecutar dinámicamente la aplicación web. Internamente, Django utiliza el estándar WSGI (Web Server Gateway Interface), que permite la comunicación entre la aplicación Python y el servidor web.



El sistema se encuentra conectado a una base de datos PostgreSQL encargada de almacenar y gestionar toda la información relacionada con usuarios, carreras, participantes, resultados y apuestas.

4 Descripción general del producto

4.1 *Visión general del sistema: límites del sistema, funcionalidades básicas, usuarios y/o otros sistemas con los que pueda interactuar.*

El sistema desarrollado consiste en una aplicación web destinada a la consulta y gestión de información relacionada con carreras de caballos, permitiendo además la realización de apuestas por parte de los usuarios registrados.

La plataforma centraliza datos relacionados con carreras, participantes, caballos, jinetes, entrenadores, hipódromos y resultados, ofreciendo una interfaz accesible desde navegador web para facilitar el seguimiento de competiciones hípcas.

El sistema diferencia distintos tipos de usuarios según las funcionalidades disponibles:

- Usuarios no registrados:
 - pueden consultar carreras,
 - visualizar participantes,
 - acceder a información básica del sistema.
- Usuarios registrados:
 - pueden iniciar sesión,
 - acceder a su perfil personal,
 - realizar apuestas,
 - consultar historial de apuestas y estadísticas relacionadas.

La aplicación interactúa directamente con una base de datos relacional encargada de almacenar toda la información del sistema. Entre las principales entidades gestionadas destacan usuarios, carreras, participantes, caballos, apuestas y resultados.

Además, el proyecto incorpora procesos automatizados de organización y tratamiento de datos procedentes de distintas fuentes relacionadas con competiciones hípcas, facilitando la actualización y estructuración de la información mostrada en la plataforma.

El sistema se ha diseñado siguiendo una arquitectura web basada en el patrón MVT (Model-View-Template), permitiendo separar correctamente la lógica de negocio, el acceso a datos y la presentación visual de la aplicación.

4.2 *Descripción breve de métodos, técnicas o arquitecturas(m/t/a) utilizadas.*

Para el desarrollo del proyecto se utilizaron diferentes métodos y técnicas orientados a garantizar una estructura organizada, escalable y fácilmente mantenible.

La aplicación se encuentra desarrollada siguiendo la arquitectura MVT (Model-View-Template), utilizada por Django. Esta arquitectura divide la aplicación en tres componentes principales:

- **Model:**
encargado de gestionar la estructura de datos y la comunicación con la base de datos.
- **View:**
responsable de procesar las peticiones realizadas por el usuario y controlar la lógica de la aplicación.
- **Template:**
encargado de la representación visual de las páginas web mediante HTML, CSS y contenido dinámico.

La base de datos fue diseñada utilizando un modelo relacional implementado sobre PostgreSQL, estableciendo relaciones entre entidades mediante claves primarias y foráneas para garantizar la integridad de la información.

También se aplicaron técnicas de validación de usuarios y autenticación mediante el sistema integrado de Django, permitiendo gestionar sesiones, control de acceso y protección de determinadas funcionalidades de la aplicación.

Por otra parte, se desarrollaron scripts en Python destinados al procesamiento y reorganización de datos relacionados con carreras hípicas, permitiendo automatizar la preparación de información antes de ser almacenada en la base de datos.

En el desarrollo de la interfaz se utilizaron tecnologías frontend como HTML, CSS y JavaScript para construir una experiencia visual clara, dinámica e intuitiva para el usuario.

4.3 Despliegue de la aplicación indicando plataforma tecnológica, instalación de la aplicación y puesta en marcha

La aplicación ha sido desarrollada utilizando Python junto con el framework Django y una base de datos PostgreSQL como sistema de almacenamiento principal.

El funcionamiento del sistema se basa en una arquitectura cliente-servidor, donde el navegador web realiza peticiones HTTP que son procesadas por el servidor de desarrollo integrado de Django. Dicho servidor utiliza el estándar WSGI (Web Server Gateway Interface), encargado de establecer la comunicación entre la aplicación desarrollada en Python y el servidor web.

La aplicación utiliza una base de datos PostgreSQL configurada específicamente para almacenar toda la información relacionada con usuarios, carreras, apuestas, participantes y resultados.

Para la puesta en funcionamiento del sistema es necesario disponer de:

- Python instalado en el sistema.
- Dependencias necesarias del proyecto.
- Servidor PostgreSQL correctamente configurado.
- Base de datos creada previamente.
- Entorno virtual de Python para aislar las librerías utilizadas.

Una vez configurado el entorno, la aplicación puede ejecutarse accediendo posteriormente desde navegador web a la dirección configurada por el servidor de Django.

El sistema ha sido desarrollado y probado en un entorno orientado al desarrollo y depuración, permitiendo realizar modificaciones y comprobar los cambios de forma inmediata durante la implementación del proyecto.

5 Planificación y presupuesto

La planificación del proyecto se realizó de forma progresiva, dividiendo el desarrollo en varias fases con el objetivo de organizar correctamente las tareas y facilitar el control del avance de la aplicación.

En una primera fase se llevó a cabo el análisis inicial del sistema, definiendo las funcionalidades principales de la plataforma, los requisitos necesarios y la estructura general de la base de datos.

Posteriormente, se desarrolló el diseño de la base de datos relacional, estableciendo las entidades principales del sistema y las relaciones existentes entre usuarios, carreras, caballos, participantes, resultados y apuestas.

Una vez diseñada la estructura de datos, se procedió a la creación del proyecto utilizando Django, implementando progresivamente las diferentes funcionalidades:

- sistema de registro e inicio de sesión,
- gestión de usuarios,
- visualización de carreras,
- consultas de participantes,
- sistema de apuestas,
- perfiles de usuario,
- estadísticas y consultas dinámicas.

De forma paralela, se desarrollaron scripts de procesamiento de datos en Python destinados a reorganizar y estructurar información relacionada con carreras hípcas antes de almacenarla en la base de datos.

Finalmente, se realizaron pruebas de funcionamiento y corrección de errores para garantizar la estabilidad y el correcto comportamiento de la aplicación.

El desarrollo del proyecto se organizó de forma progresiva a lo largo de varias fases. En primer lugar, se realizó el análisis inicial y la planificación general del sistema, definiendo las funcionalidades principales de la aplicación y los requisitos necesarios para su implementación.

Posteriormente, se llevó a cabo el diseño de la base de datos relacional, estableciendo las entidades y relaciones necesarias para gestionar usuarios, carreras, caballos, participantes, resultados y apuestas. Una vez definida esta estructura, se configuró el entorno de desarrollo y se creó la base inicial del proyecto utilizando Django.

La mayor parte del tiempo de desarrollo se centró en la implementación del backend y de las distintas funcionalidades de la aplicación, incluyendo la gestión de usuarios, el sistema de autenticación, la visualización de carreras y participantes, así como el sistema de apuestas y perfiles de usuario.

Además, se desarrollaron las vistas e interfaces necesarias para facilitar la interacción con el sistema, junto con scripts en Python destinados al procesamiento y reorganización automática de datos relacionados con carreras hípcas.

Finalmente, se dedicó una fase específica a la realización de pruebas, detección de errores y validación general del funcionamiento de la aplicación, así como a la elaboración de la documentación y memoria final del proyecto.

La planificación se realizó de manera flexible, permitiendo adaptar tiempos y prioridades en función de las necesidades y problemas detectados durante el desarrollo.

Aunque el proyecto ha sido desarrollado con fines académicos y sin coste económico directo, es posible realizar una estimación aproximada del presupuesto teniendo en cuenta el tiempo de desarrollo y las herramientas utilizadas.

La mayoría de tecnologías empleadas son de código abierto y gratuitas, como:

- Python,
- Django,
- PostgreSQL,
- y Visual Studio Code.

La estimación económica principal estaría asociada al tiempo invertido en el desarrollo del proyecto.

Concepto	Coste estimado
Desarrollo y programación	2.500 €
Diseño y pruebas	500 €
Elaboración de documentación	300 €
Infraestructura y herramientas	0 €
Total aproximado	3.300 €

El presupuesto mostrado representa únicamente una estimación orientativa del valor del trabajo realizado si el proyecto hubiese sido desarrollado en un entorno profesional.

6 Documentación Técnica: análisis, diseño, implementación y pruebas.

6.1 Especificación de requisitos

Antes de comenzar el desarrollo de la aplicación se realizó un análisis de requisitos con el objetivo de identificar las funcionalidades necesarias y determinar el comportamiento general del sistema.

La aplicación debía permitir gestionar información relacionada con carreras de caballos, participantes, resultados y apuestas, proporcionando además un sistema de autenticación para usuarios registrados.

Entre los principales requisitos funcionales definidos para el proyecto destacan:

- Registro e inicio de sesión de usuarios.
- Visualización de carreras y participantes.
- Consulta de información relacionada con caballos, jinetes y entrenadores.
- Sistema de apuestas asociado a cada carrera.
- Gestión de perfiles de usuario.
- Consulta de estadísticas e historial de apuestas.
- Almacenamiento persistente de información en base de datos.

Además, también se definieron diferentes requisitos no funcionales relacionados con:

- rendimiento,
- organización del código,
- integridad de datos,
- facilidad de mantenimiento,
- y usabilidad de la interfaz.

El sistema debía garantizar una correcta relación entre las distintas entidades de la base de datos, evitando duplicidades y manteniendo la coherencia de la información almacenada.

6.2 Análisis del sistema

Durante la fase de análisis se estudió la estructura general del sistema y las relaciones necesarias entre los diferentes componentes de la aplicación.

La plataforma se diseñó siguiendo una arquitectura web cliente-servidor, donde los usuarios interactúan con la aplicación mediante navegador web mientras el servidor procesa las peticiones y realiza consultas a la base de datos.

A nivel interno, el sistema se encuentra dividido en varias áreas funcionales:

- Gestión de usuarios.
- Gestión de carreras.
- Gestión de participantes y resultados.
- Sistema de apuestas.
- Procesamiento de datos externos.
- Gestión de estadísticas y perfiles.

El análisis permitió definir las entidades principales del proyecto y sus relaciones dentro de la base de datos. Entre las tablas más importantes se encuentran usuarios, carreras, caballos, participantes, resultados, apuestas, jinetes, entrenadores e hipódromos.

También se analizaron las restricciones necesarias para garantizar la integridad de los datos, utilizando claves primarias, claves foráneas y restricciones de unicidad entre distintas tablas.

Por otra parte, se definió el flujo general de funcionamiento de la aplicación, estableciendo cómo interactúan los usuarios con el sistema y cómo se gestionan las operaciones relacionadas con autenticación, consultas y apuestas.

6.3 Diseño del sistema:

6.3.1 Diseño de la Base de Datos

Durante la fase de análisis se estudió la estructura general del sistema y las relaciones necesarias entre los diferentes componentes de la aplicación.

La plataforma se diseñó siguiendo una arquitectura web cliente-servidor, donde los usuarios interactúan con la aplicación mediante navegador web mientras el servidor procesa las peticiones y realiza consultas a la base de datos.

A nivel interno, el sistema se encuentra dividido en varias áreas funcionales:

- Gestión de usuarios.
- Gestión de carreras.
- Gestión de participantes y resultados.
- Sistema de apuestas.
- Procesamiento de datos externos.
- Gestión de estadísticas y perfiles.

El análisis permitió definir las entidades principales del proyecto y sus relaciones dentro de la base de datos. Entre las tablas más importantes se encuentran usuarios, carreras, caballos, participantes, resultados, apuestas, jinetes, entrenadores e hipódromos.

También se analizaron las restricciones necesarias para garantizar la integridad de los datos, utilizando claves primarias, claves foráneas y restricciones de unicidad entre distintas tablas.

Por otra parte, se definió el flujo general de funcionamiento de la aplicación, estableciendo cómo interactúan los usuarios con el sistema y cómo se gestionan las operaciones relacionadas con autenticación, consultas y apuestas.

Además del diseño manual de las tablas principales, Django incorpora automáticamente determinadas funcionalidades adicionales dentro de la base de datos y del sistema de autenticación. Entre ellas destacan:

- la gestión interna de usuarios mediante la tabla `auth_user`,
- el control de permisos y grupos,
- la gestión de sesiones,
- y el sistema de migraciones encargado de crear y actualizar automáticamente la estructura de la base de datos.

En este proyecto, el sistema de autenticación integrado de Django se complementó con una tabla personalizada de usuarios que almacena información adicional como apellidos, fecha de nacimiento y DNI, ampliando así las funcionalidades básicas proporcionadas por el framework.

Por otra parte, la implementación mediante modelos de Django facilitó enormemente la realización de consultas complejas utilizando el ORM, permitiendo acceder a relaciones entre tablas de forma más sencilla, segura y organizada sin necesidad de escribir grandes cantidades de SQL manual.

Finalmente, el diseño relacional adoptado permite mantener una estructura modular, organizada y escalable, facilitando futuras ampliaciones del sistema y mejorando el mantenimiento general de la aplicación.

6.3.2 Diseño de la Interfaz de usuario.

La interfaz de usuario fue diseñada con el objetivo de proporcionar una experiencia visual clara, sencilla e intuitiva.

Para el desarrollo de las interfaces se utilizaron tecnologías como HTML, CSS y JavaScript, permitiendo crear páginas dinámicas adaptadas a la navegación web.

La estructura visual de la aplicación sigue una organización basada en menús de navegación, tarjetas informativas y secciones diferenciadas para facilitar el acceso a las distintas funcionalidades del sistema.

Entre las principales interfaces desarrolladas destacan:

- Página principal de carreras.
- Pantalla de inicio de sesión y registro.
- Perfil de usuario.
- Visualización de participantes y resultados.
- Panel de apuestas y estadísticas.
- Interfaz de administración del sistema.

Además de las interfaces orientadas al usuario final, el proyecto también utiliza la interfaz de administración integrada proporcionada por Django. Esta herramienta permite gestionar

de forma sencilla toda la información almacenada en la base de datos mediante un panel administrativo accesible únicamente para usuarios con permisos de administrador.

La interfaz de administración facilita operaciones como:

- creación, modificación y eliminación de registros,
- gestión de carreras y participantes,
- administración de usuarios,
- consulta de apuestas,
- y mantenimiento general de la información del sistema.

Gracias al sistema administrativo de Django, fue posible gestionar gran parte de los datos del proyecto sin necesidad de desarrollar paneles personalizados adicionales, agilizando tanto las tareas de desarrollo como las pruebas y validaciones de la aplicación.

Asimismo, Django incorpora automáticamente funcionalidades relacionadas con autenticación, control de permisos y gestión de sesiones dentro de esta interfaz administrativa, proporcionando un entorno seguro y organizado para la administración interna del sistema.

6.3.3 Diseño de la Aplicación.

La aplicación fue desarrollada siguiendo la arquitectura MVT (Model-View-Template) proporcionada por Django. Esta arquitectura permite organizar el proyecto de forma estructurada, separando correctamente la lógica de negocio, el acceso a datos y la representación visual de la aplicación.

El patrón MVT utilizado por Django guarda una gran similitud con la arquitectura MVC (Model-View-Controller), ampliamente utilizada en el desarrollo de aplicaciones web. Ambas arquitecturas comparten el objetivo principal de separar responsabilidades dentro de la aplicación para facilitar el mantenimiento, reutilización y escalabilidad del código.

En ambos modelos:

- el componente **Model** se encarga de gestionar los datos y la comunicación con la base de datos,
- y la parte visual de la aplicación se encuentra separada de la lógica principal del sistema.

Sin embargo, existen algunas diferencias entre ambas arquitecturas. En el patrón MVC tradicional:

- la **View** representa la interfaz visual mostrada al usuario,
- mientras que el **Controller** procesa las peticiones y controla la lógica de funcionamiento.

Por el contrario, en la arquitectura MVT de Django:

- las **Templates** son las encargadas de representar la interfaz visual,
- mientras que las **Views** asumen parte del comportamiento que en MVC correspondería al Controller.

Además, Django incorpora internamente numerosos mecanismos automáticos que simplifican el desarrollo de aplicaciones web, como:

- sistema de rutas,
- autenticación de usuarios,
- panel de administración,
- ORM para bases de datos,
- gestión de sesiones,
- y sistema de migraciones.

La arquitectura MVT permitió dividir la aplicación en tres componentes principales:

- **Model:**
encargado de definir la estructura de datos y gestionar la comunicación con la base de datos mediante modelos relacionales.
- **View:**
responsable de procesar las peticiones realizadas por el usuario, controlar la lógica de negocio y enviar la información correspondiente a las plantillas.
- **Template:**
encargado de representar dinámicamente la información mostrada en el navegador utilizando HTML, CSS y contenido generado desde el servidor.

La organización del proyecto se realizó separando modelos, vistas, rutas, plantillas y archivos estáticos, facilitando el mantenimiento y la ampliación futura del sistema.

Además, se implementaron mecanismos de autenticación utilizando el sistema integrado de Django, permitiendo controlar el acceso a determinadas funcionalidades mediante sesiones de usuario y restricciones para usuarios no autenticados.

Otro aspecto relevante dentro del diseño de la aplicación fue el uso del ORM de Django, el cual permitió realizar consultas a la base de datos utilizando objetos y modelos de Python en lugar de escribir directamente consultas SQL complejas. Esto facilitó enormemente el desarrollo y mejoró la organización general del código.

Asimismo, el proyecto incorpora scripts auxiliares en Python destinados al procesamiento y reorganización de datos relacionados con carreras hípias antes de ser almacenados en la base de datos, automatizando parte del tratamiento de información utilizado por la aplicación.

6.4 Implementación:

6.4.1 Entorno de desarrollo.

El desarrollo del proyecto se realizó utilizando Python como lenguaje principal junto con el framework Django.

La aplicación utiliza el servidor de desarrollo integrado de Django, encargado de gestionar las peticiones HTTP realizadas desde el navegador y ejecutar dinámicamente la aplicación web mediante el estándar WSGI.

Como sistema gestor de base de datos se utilizó PostgreSQL, encargado de almacenar toda la información relacionada con usuarios, carreras, apuestas y participantes.

Además, se emplearon herramientas como Visual Studio Code, Git y entornos virtuales de Python para facilitar el desarrollo y la gestión del proyecto.

6.4.2 Estructura del código.

La organización del código sigue la estructura estándar de proyectos desarrollados con Django.

El proyecto se encuentra dividido en diferentes módulos encargados de gestionar funcionalidades específicas del sistema, separando correctamente vistas, modelos, URLs, plantillas y archivos estáticos.

Dentro de la aplicación se implementaron:

- modelos encargados de representar las tablas de la base de datos,
- vistas responsables de procesar las peticiones,
- plantillas HTML para la representación visual,
- y rutas URL destinadas a conectar las distintas funcionalidades de la aplicación.

Además, se desarrollaron scripts independientes para el procesamiento y reorganización de datos relacionados con carreras hípias.

Esta estructura modular facilita el mantenimiento, reutilización y ampliación futura del sistema.

6.4.3 Cuestiones de diseño e implementación reseñables.

Durante el desarrollo del proyecto surgieron diferentes aspectos técnicos relevantes relacionados con autenticación, relaciones entre entidades y gestión de datos.

Uno de los principales retos consistió en diseñar correctamente la estructura relacional de la base de datos, garantizando la integridad entre carreras, participantes, caballos y apuestas.

También fue necesario implementar mecanismos de control de acceso utilizando el sistema de autenticación de Django, restringiendo determinadas funcionalidades únicamente a usuarios registrados.

Otro aspecto destacable fue el procesamiento automático de datos mediante scripts en Python, permitiendo reorganizar información procedente de distintas fuentes antes de almacenarla en la base de datos.

Además, se optimizaron consultas utilizando relaciones entre modelos y consultas ORM proporcionadas por Django, mejorando el rendimiento general de determinadas operaciones dentro de la aplicación.

6.5 Pruebas.

Durante el desarrollo del proyecto se realizaron diferentes pruebas funcionales y de validación con el objetivo de garantizar el correcto funcionamiento del sistema.

Las pruebas realizadas permitieron comprobar:

- el funcionamiento del registro e inicio de sesión,
- la navegación entre páginas,
- la visualización correcta de carreras y participantes,
- el sistema de apuestas,
- las relaciones entre entidades de la base de datos,
- y el correcto almacenamiento de información.

También se realizaron pruebas relacionadas con validación de formularios, control de errores y restricciones de acceso para usuarios no autenticados.

Además, se comprobaron consultas y operaciones sobre la base de datos para garantizar la integridad de los datos y evitar inconsistencias entre las distintas tablas del sistema.

Estas pruebas permitieron detectar y corregir errores durante el desarrollo, mejorando la estabilidad y fiabilidad general de la aplicación.

7 Manuales de usuario

7.1 Manual de usuario

1. Interfaz de Usuario

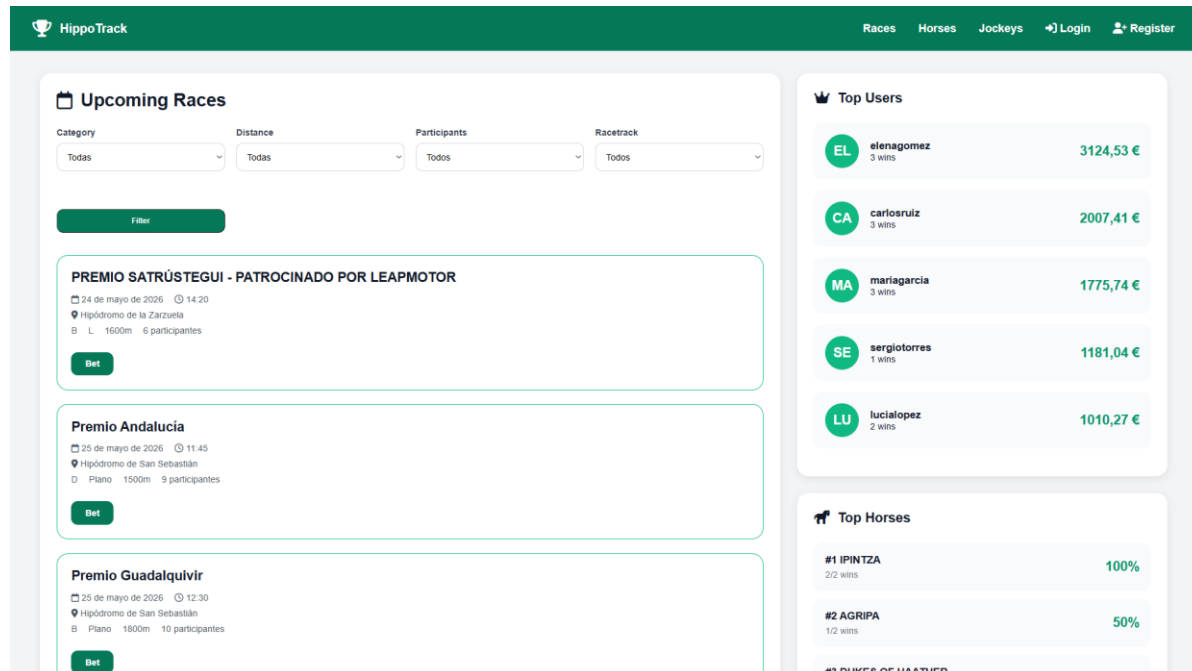
La plataforma HippoTrack permite a los usuarios consultar carreras, caballos y jinetes, además de realizar apuestas y gestionar su perfil personal de forma intuitiva.

1.1 Página principal

La página principal muestra las próximas carreras disponibles, filtros de búsqueda y estadísticas destacadas de usuarios y caballos.

Características principales:

- Filtros por categoría, distancia, participantes e hipódromo.
- Listado de carreras disponibles para apostar.
- Ranking de mejores usuarios.
- Ranking de caballos con mejores resultados.



The screenshot displays the HippoTrack website interface. The header is green with the HippoTrack logo and navigation links: Races, Horses, Jockeys, Login, and Register. The main content area is divided into three sections:

- Upcoming Races:** Features filters for Category, Distance, Participants, and Racetrack. Below the filters, there are three race listings:
 - PREMIO SATRÚSTEGUI - PATROCINADO POR LEAPMOTOR:** 24 de mayo de 2026, 14:20, Hipódromo de la Zarzuela, 1600m, 6 participantes. Bet button.
 - Premio Andalucía:** 25 de mayo de 2026, 11:45, Hipódromo de San Sebastián, Plano, 1500m, 9 participantes. Bet button.
 - Premio Guadalquivir:** 25 de mayo de 2026, 12:30, Hipódromo de San Sebastián, Plano, 1800m, 10 participantes. Bet button.
- Top Users:** A list of top users with their initials, names, wins, and earnings:
 - EL elenagomez, 3 wins, 3124,53 €
 - CA carlosruiz, 3 wins, 2007,41 €
 - MA mariagarcia, 3 wins, 1775,74 €
 - SE sergiotorres, 1 wins, 1181,04 €
 - LU lucialopez, 2 wins, 1010,27 €
- Top Horses:** A list of top horses with their names, wins, and win percentages:
 - #1 IPINTZA, 2/2 wins, 100%
 - #2 AGRIPA, 1/2 wins, 50%
 - #3 DUKES OF HAATHER, 0/0 wins, 0%

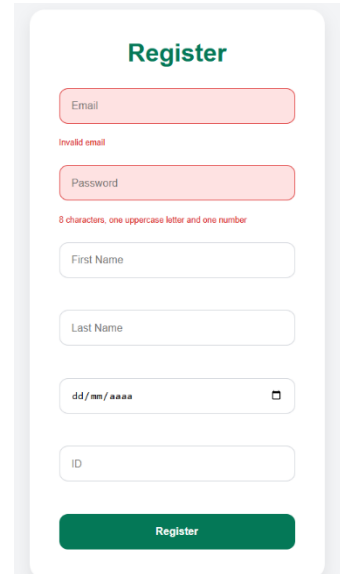
1.2 Registro de usuario

Los nuevos usuarios pueden crear una cuenta introduciendo:

- Correo electrónico.
- Contraseña.
- Nombre y apellidos.
- Fecha de nacimiento.
- Documento identificativo.

El sistema valida automáticamente:

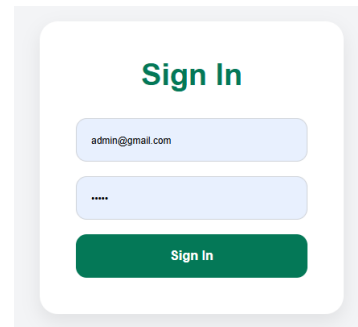
- Formato del email.
- Fortaleza de la contraseña.



The Register form is a vertical stack of input fields. At the top is a green 'Register' header. Below it are fields for Email, Password, First Name, Last Name, and a date field (dd/mm/yyyy). The Password field has a red error message: 'Invalid email' and '8 characters, one uppercase letter and one number'. At the bottom is a green 'Register' button.

1.3 Inicio de sesión

Los usuarios registrados pueden acceder a la plataforma mediante su correo y contraseña.



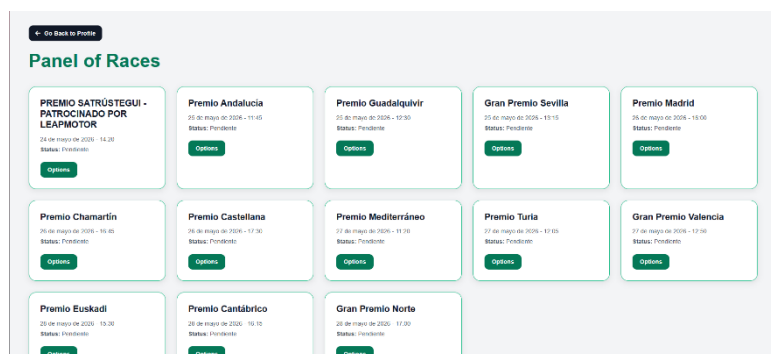
The Sign In form is a vertical stack of input fields. It has a green 'Sign In' header. Below it are fields for email (admin@gmail.com) and password (*****). At the bottom is a green 'Sign In' button.

1.4 Listado de carreras

En esta sección se muestran todas las carreras disponibles.

Cada tarjeta incluye:

- Nombre de la carrera.
- Fecha y hora.
- Estado actual.
- Botón de acceso a opciones o apuestas.



The Panel of Races is a grid of 12 race cards. Each card has a title, date and time, status, and an 'Options' button. The cards are arranged in a 3x4 grid.

Race Name	Date and Time	Status
PREMIO SATRÚSTEGUI - PATROCINADO POR LEAPMOTOR	24 de mayo de 2020 - 14:30	Pendiente
Premio Andalucía	25 de mayo de 2020 - 11:45	Pendiente
Premio Guadalquivir	26 de mayo de 2020 - 12:30	Pendiente
Gran Premio Sevilla	26 de mayo de 2020 - 15:15	Pendiente
Premio Madrid	26 de mayo de 2020 - 16:00	Pendiente
Premio Chamartín	26 de mayo de 2020 - 16:45	Pendiente
Premio Castellana	26 de mayo de 2020 - 17:30	Pendiente
Premio Mediterráneo	27 de mayo de 2020 - 11:00	Pendiente
Premio Turia	27 de mayo de 2020 - 11:15	Pendiente
Gran Premio Valencia	27 de mayo de 2020 - 12:30	Pendiente
Premio Euskadi	28 de mayo de 2020 - 13:30	Pendiente
Premio Cantábrico	28 de mayo de 2020 - 16:15	Pendiente
Gran Premio Norte	28 de mayo de 2020 - 17:00	Pendiente

1.5 Pantalla de apuestas

Los usuarios pueden realizar apuestas sobre los participantes de una carrera.

Información disponible:

- Caballo participante.
- Jinete.
- Peso.
- Cuota.
- Tipo de apuesta.
- Cantidad apostada.

También se muestra el total apostado acumulado.

PREMIO SATRÚSTEGUI - PATROCINADO POR LEAPMOTOR

Racetrack: Hipódromo de la Zarzuela

Date: 24 de mayo de 2026

Time: 14:20

Distance: 1600m

Type: L

Category: B

Track type:

Track condition:

Status: Pendiente

Total Bet

455,60 €

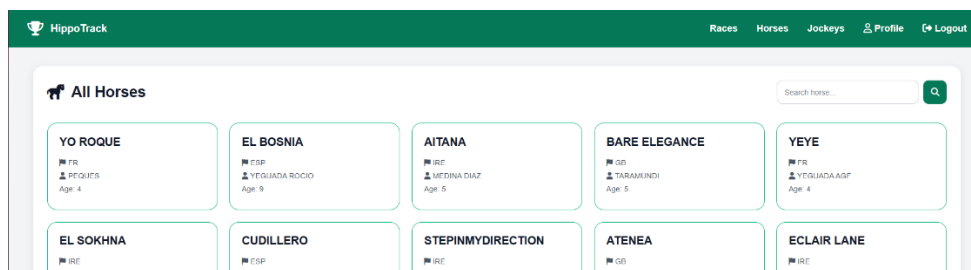
#	Horse	Jockey	Weight	Odds	Bet
3	MAXIMUM SCEPTICISM	J. GELABERT	61.00 kg	x19.08	Generate Bet
6	ZODIACO	R. N. VALLE	60.00 kg	x16.42	Generate Bet
1	JUAN DE AUSTRIA	D. SKOROVÁ	58.50 kg	x2.66	Generate Bet
2	MARCUS ATILIUS	I. MELGAREJO	56.00 kg	x7.35	Generate Bet
5	SKY RIM	R. SOUSA	58.50 kg	x9.43	Generate Bet
4	RENOVATIO ANGEL	M. MONTERISO	57.00 kg	x5.36	Generate Bet

1.6 Listado de caballos

Permite consultar todos los caballos registrados en el sistema.

Funciones:

- Buscador de caballos.
- Información básica de cada caballo:
 - Nacionalidad.
 - Propietario.
 - Edad.

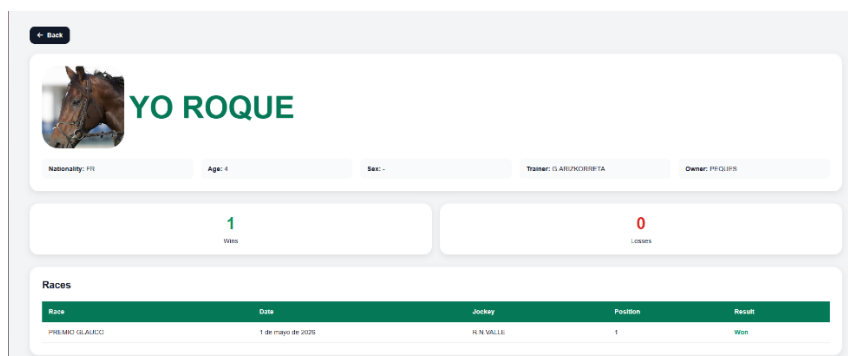


1.7 Vista detallada de caballo

Muestra toda la información y estadísticas de un caballo específico.

Información incluida:

- Imagen del caballo.
- Nacionalidad.
- Edad.
- Sexo.
- Entrenador.
- Propietario.
- Número de victorias y derrotas.
- Historial de carreras.

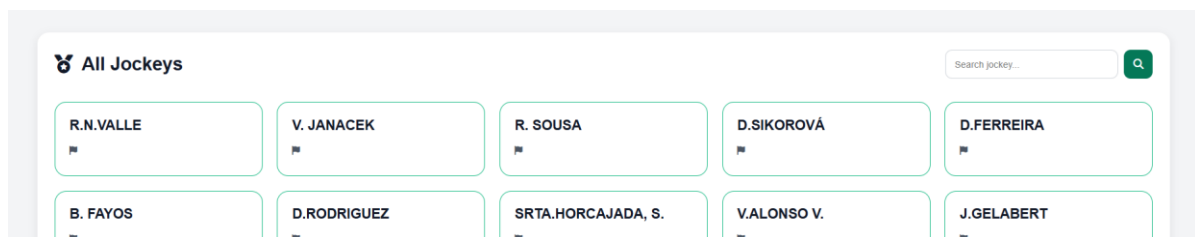


1.8 Listado de jinetes

Permite consultar todos los jinetes registrados.

Funciones:

- Buscador de jinetes.
- Consulta rápida de nombres y nacionalidad.

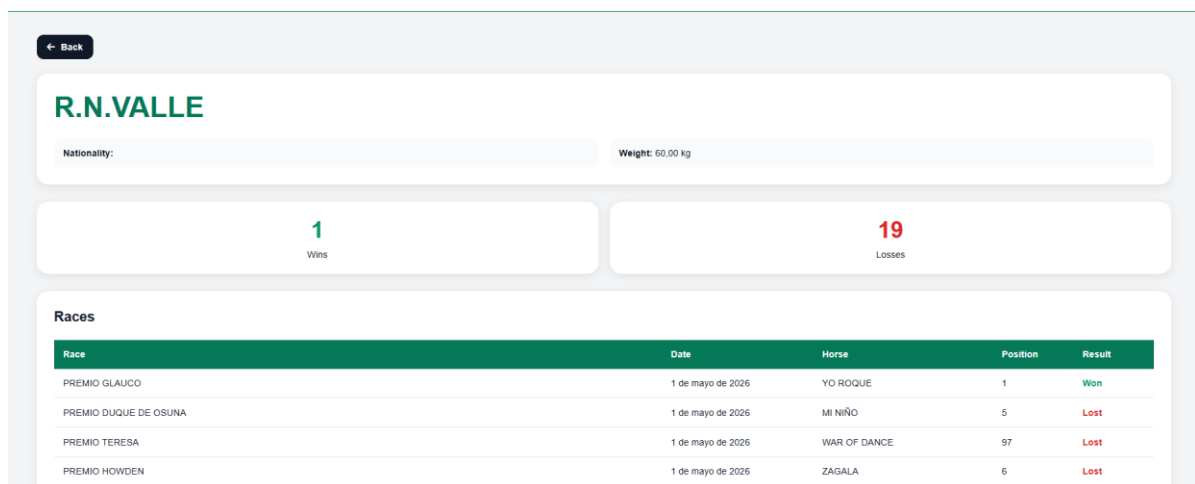


1.9 Vista detallada de jinete

Muestra las estadísticas completas de un jinete.

Información disponible:

- Nacionalidad.
- Peso.
- Victorias y derrotas.
- Historial de carreras disputadas.

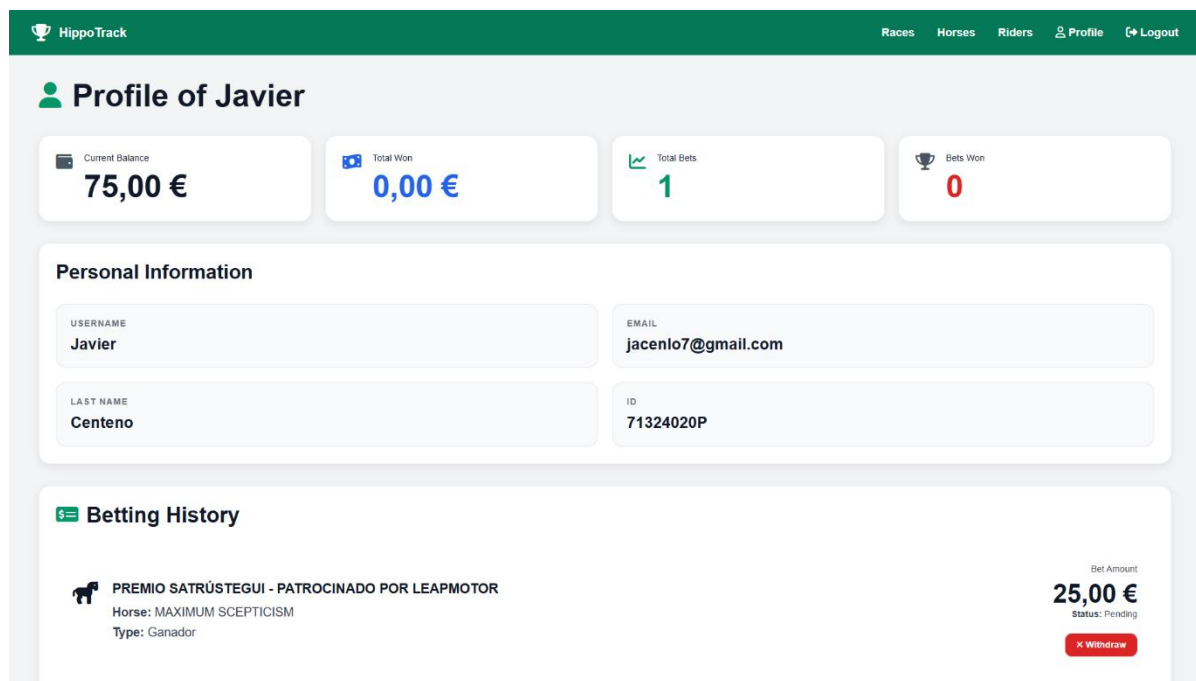


1.10 Perfil de usuario

Cada usuario dispone de un perfil personal con:

- Balance disponible.
- Ganancias totales.
- Número de apuestas realizadas.
- Apuestas ganadas.
- Historial completo de apuestas.

Además, desde esta sección es posible retirar apuestas pendientes.



The screenshot shows the user profile for 'Javier'. At the top, there's a green navigation bar with 'HippoTrack' and links for 'Races', 'Horses', 'Riders', 'Profile', and 'Logout'. The profile section is titled 'Profile of Javier' and includes four summary cards: 'Current Balance' (75,00 €), 'Total Won' (0,00 €), 'Total Bets' (1), and 'Bets Won' (0). Below these is the 'Personal Information' section with fields for Username (Javier), Email (jacenlo7@gmail.com), Last Name (Centeno), and ID (71324020P). The 'Betting History' section shows a single bet for 'PREMIO SATRÚSTEGUI - PATROCINADO POR LEAPMOTOR' on the horse 'MAXIMUM SCEPTICISM', with a bet amount of 25,00 € and a status of 'Pending'. A red 'Withdraw' button is visible next to the pending bet.

2. Interfaz de Administrador

La interfaz de administrador permite gestionar las carreras y supervisar la información del sistema.

2.1 Perfil del administrador

El administrador dispone de un perfil similar al usuario estándar, pero con acceso a herramientas de gestión.

Funciones adicionales:

- Acceso al panel de administración de carreras.
- Gestión general de la plataforma.

 **Profile of admin**

 Current Balance
75,00 €

 Total Won
0,00 €

 Total Bets
1

 Bets Won
0

Personal Information

USERNAME
admin

EMAIL
admin@gmail.com

LAST NAME
Administrador

ID
TEMP-1

Administration

 **Manage Races**

2.2 Panel de configuración de carreras

Desde este panel el administrador puede:

- Finalizar carreras manualmente.
- Generar resultados aleatorios.
- Añadir apuestas automáticamente.
- Consultar apuestas realizadas.
- Visualizar cuotas y participantes.

Cada participante muestra:

- Nombre del caballo.
- Jinete.
- Peso.
- Lista de apuestas asociadas.

 Back to panel

PREMIO SATRÚSTEGUI - PATROCINADO POR LEAPMOTOR

Racecourse: Hipódromo de la Zarzosa Date: 24 de mayo de 2026 Time: 14:20 Distance: 1600m Category: B

Status: Pendiente

Finish Race

Random Result

Number of bettors

Add bets

Total Wagered
505,60 €

#3 MAXIMUM SCEPTICISM **x5,51**

Jockey: J. GELABERT
Weight: 61.00 kg

Bets

carlosruiz 45,21 € Exacta

Javier 25,00 € Ganador

admin 25,00 € Ganador

#6 ZODIACO **x11,57**

Jockey: R.N. VALLE
Weight: 60.00 kg

Bets

lucialopez 43,71 € Ganador

#1 JUAN DE AUSTRIA **x2,85**

Jockey: D. SIKOROVÁ
Weight: 58.50 kg

Bets

juanperez 171,28 € Ganador

#2 MARCUS ATILIUS **x8,15**

Jockey: I. MELGAREJO
Weight: 56.00 kg

#5 SKY RIM **x10,46**

Jockey: R. SOUSA
Weight: 58.50 kg

#4 RENOVATIO ANGEL **x5,94**

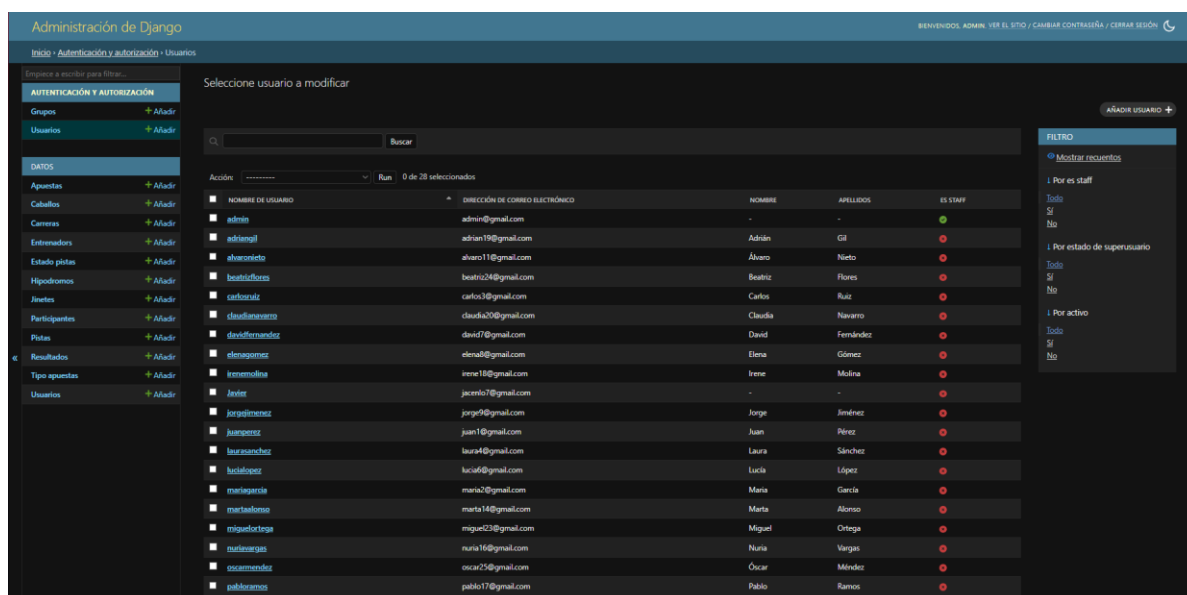
Jockey: M. MONTERISO
Weight: 57.00 kg

2.3 Panel de administración Django

La plataforma incorpora el panel administrativo de Django para la gestión completa de la base de datos.

Desde este panel se pueden administrar:

- Usuarios.
- Caballos.
- Carreras.
- Participantes.
- Jinetes.
- Hipódromos.
- Apuestas.
- Resultados.
- Tipos de apuestas.



The screenshot shows the Django Admin interface for the 'Usuarios' (Users) section. The sidebar on the left contains navigation links for 'Inicio', 'Autenticación y autorización', 'Usuarios', 'Apuestas', 'Caballos', 'Carreras', 'Participantes', 'Jinetes', 'Hipódromos', 'Apuestas', 'Resultados', 'Tipos de apuestas', and 'Usuarios'. The main content area displays a table of users with columns for 'Nombre de usuario', 'Dirección de correo electrónico', 'Nombre', 'Apellido', and 'Es staff'. The table lists 17 users, including 'admin', 'adrian', 'alvaro', 'beatriz', 'carlos', 'claudia', 'david', 'elena', 'jorge', 'juan', 'laura', 'lucia', 'maria', 'marta', 'miguel', 'nuria', 'oscar', and 'pablo'. The 'Es staff' column indicates whether the user is a staff member, with a green dot for 'admin' and a red dot for others.

3. Base de datos

La aplicación utiliza una base de datos relacional en PostgreSQL que gestiona usuarios, carreras, participantes, apuestas y resultados.

7.2 Manual de instalación

Para ejecutar correctamente la aplicación es necesario disponer de:

- Python instalado.
- PostgreSQL instalado y configurado.
- Acceso a terminal o línea de comandos.
- El repositorio del proyecto descargado.

1. Descargar el proyecto

El primer paso consiste en descargar o clonar el repositorio del proyecto y acceder a la carpeta principal desde terminal.

```
git clone https://github.com/cookedpatata/TFG.git  
cd nombre_del_proyecto
```

2. Crear el entorno virtual

Se recomienda crear un entorno virtual de Python para aislar las dependencias del proyecto y evitar conflictos con otras librerías instaladas en el sistema.

- **Crear entorno virtual**

```
python -m venv venv
```

- **Activar entorno virtual**
- **Windows**

```
venv\Scripts\activate
```

- **Linux / MacOS**

```
source venv/bin/activate
```

3. Instalar dependencias

Con el entorno virtual activado, se deben instalar todas las dependencias necesarias del proyecto mediante el archivo requirements.txt.

```
pip install -r requirements.txt
```

Este archivo contiene todas las librerías utilizadas durante el desarrollo de la aplicación.

4. Crear la base de datos

Antes de ejecutar el proyecto es necesario instalar PostgreSQL versión 17, utilizada durante el desarrollo de la aplicación.

Descarga oficial:

<https://www.postgresql.org/download/>

Una vez instalado PostgreSQL, debe crearse una base de datos destinada exclusivamente al proyecto.

La estructura completa de la base de datos se encuentra incluida dentro del archivo SQL proporcionado junto al proyecto (BD.sql).

Este archivo contiene:

- creación de tablas,
- relaciones,
- claves foráneas,
- restricciones,
- y estructura general del sistema.

Para generar toda la estructura de la base de datos, únicamente es necesario ejecutar el archivo SQL sobre PostgreSQL utilizando una herramienta compatible como:

- pgAdmin,
- consola psql,
- o cualquier cliente SQL compatible con PostgreSQL.

La base de datos almacenará toda la información relacionada con:

- usuarios,
 - carreras,
 - participantes,
 - caballos,
 - apuestas,
 - resultados,
 - y demás entidades del sistema.
-

5. Configurar settings.py

Después de crear la base de datos, es necesario modificar la configuración de conexión dentro del archivo settings.py de Django.

En el apartado DATABASES deben configurarse:

- nombre de la base de datos,
- usuario,
- contraseña,
- host,
- y puerto.

Ejemplo de configuración:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'TFG',  
        'USER': 'postgres',  
        'PASSWORD': 'contraseña',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

Además, dependiendo del entorno donde se ejecute la aplicación, también puede ser necesario modificar:

- ALLOWED_HOSTS,
- DEBUG,
- configuración de archivos estáticos,
- y rutas de plantillas.

6. Ejecutar migraciones

Una vez configurada la base de datos, deben ejecutarse las migraciones de Django.

- **Crear migraciones**

```
python manage.py makemigrations
```

- **Aplicar migraciones**

```
python manage.py migrate
```

El sistema de migraciones se encarga de crear automáticamente todas las tablas necesarias dentro de PostgreSQL a partir de los modelos definidos en la aplicación.

7. Crear superusuario

Tras completar las migraciones, debe crearse un superusuario de Django.

```
python manage.py createsuperuser
```

Durante la creación del superusuario se solicitarán:

- nombre de usuario,
- correo electrónico,
- y contraseña.

8. Acceder al panel de administración

Una vez iniciado el proyecto, el panel administrativo estará disponible mediante la ruta:

/admin

Desde esta interfaz es posible:

- gestionar usuarios,
- administrar carreras,
- modificar participantes,
- consultar apuestas,
- y controlar gran parte de la información almacenada en el sistema.

La interfaz administrativa de Django incorpora automáticamente:

- autenticación,
- gestión de permisos,
- sesiones,
- y administración de modelos.

9. Ejecutar la aplicación

Finalmente, la aplicación puede iniciarse utilizando el servidor de desarrollo integrado de Django.

```
python manage.py runserver
```

Una vez iniciado el servidor, la aplicación estará disponible desde navegador web y lista para su utilización.

8 Conclusiones y posibles ampliaciones

El desarrollo de este proyecto ha permitido aplicar de forma práctica gran parte de los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Web y durante el periodo de formación en empresa, integrando tecnologías actuales utilizadas en el desarrollo profesional de aplicaciones web.

A lo largo del proyecto se han trabajado aspectos relacionados con:

- desarrollo backend,
- diseño de bases de datos relacionales,
- autenticación de usuarios,
- arquitectura web,
- procesamiento de datos,
- y desarrollo de interfaces dinámicas.

La utilización de Django como framework principal permitió acelerar considerablemente el desarrollo gracias a herramientas integradas como el ORM, el sistema de autenticación, las migraciones y el panel administrativo. Además, el uso de PostgreSQL facilitó la creación de una base de datos robusta y correctamente estructurada para gestionar toda la información relacionada con carreras, participantes y apuestas.

Uno de los aspectos más importantes del proyecto fue el diseño de una estructura relacional capaz de gestionar correctamente las relaciones entre usuarios, caballos, carreras, participantes y resultados, manteniendo la integridad de los datos y permitiendo realizar consultas complejas de forma eficiente.

También resultó especialmente relevante la automatización del procesamiento de datos mediante scripts desarrollados en Python, permitiendo reorganizar información procedente de distintas fuentes antes de almacenarla dentro del sistema.

A nivel personal y profesional, el proyecto ha permitido mejorar habilidades relacionadas con:

- organización y planificación de proyectos,
- resolución de problemas,
- depuración de errores,
- estructuración de aplicaciones web,
- y trabajo con tecnologías utilizadas actualmente en entornos profesionales.

Aunque la aplicación desarrollada cumple correctamente los objetivos principales planteados inicialmente, el sistema podría ampliarse en el futuro incorporando nuevas funcionalidades y mejoras.

Entre las posibles ampliaciones futuras destacan:

- Implementación de apuestas en tiempo real.
- Desarrollo de estadísticas avanzadas y predicciones.

- Incorporación de gráficos dinámicos y análisis visual de datos.
- Sistema de notificaciones para usuarios.
- Mejora de la seguridad y validación de operaciones.
- Adaptación completa para dispositivos móviles.
- Despliegue en un entorno de producción real utilizando servidores dedicados.
- Incorporación de pasarelas de pago para apuestas reales.
- Desarrollo de paneles administrativos personalizados más avanzados.

En conclusión, el proyecto ha permitido desarrollar una aplicación web completa y funcional, integrando diferentes áreas del desarrollo web dentro de un único sistema y consolidando los conocimientos adquiridos tanto durante la formación académica como en el entorno profesional.

9 Bibliografía

Durante el desarrollo del proyecto se utilizaron diferentes recursos técnicos y documentación oficial relacionados con las tecnologías empleadas en la aplicación.

Documentación oficial

- [Documentación oficial de Django](#)
- [Documentación oficial de PostgreSQL](#)
- [Documentación oficial de Python](#)
- [Documentación oficial de Selenium](#)

Herramientas utilizadas

- [Visual Studio Code](#)
- [GitHub](#)
- [PostgreSQL Downloads](#)

Recursos de aprendizaje y apoyo

- Apuntes y materiales proporcionados durante el CFGS de Desarrollo de Aplicaciones Web.
- Documentación y ejemplos consultados durante el periodo de formación en empresa.
- Foros y recursos técnicos relacionados con desarrollo web, bases de datos, automatización y Django.

Recursos propios del proyecto

- Scripts de procesamiento y reorganización de datos desarrollados en Python.
- Modelos y estructura de base de datos diseñados específicamente para la aplicación.
- Código fuente completo desarrollado para la plataforma web de gestión y apuestas hípcas.